

DOCUMENTO DE ARQUITECTURA

Vistas del sistema y registro de decisiones de arquitectura

Proyecto: Plataforma de control de suscripciones de pago

Carrera: Ingeniería Civil Informática

Institución: Universidad Católica de Temuco

Decisiones registradas: ADR-001 a ADR-006

Fecha: 31 de agosto de 2026

Índice

Índice	2
1. Introducción	3
1.1 Qué es un ADR.....	3
1.2 Índice de decisiones	3
2. Vistas de la arquitectura	4
2.1 Vista de componentes.....	4
2.2 Vista de despliegue	4
2.3 Vista funcional	5
3. Registro de decisiones de arquitectura	6
3.1 ADR-001 · Arquitectura de microservicios	6
3.2 ADR-002 · Un esquema por servicio, no una base por servicio	8
3.3 ADR-003 · Comunicación síncrona y asíncrona	9
3.4 ADR-004 · Autenticación y propagación del contexto multi-tenant	10
3.5 ADR-005 · Origen de los datos de cobros	12
3.6 ADR-006 · Dos clientes: aplicación móvil y aplicación web	18
4. Trazabilidad y mantención.....	22
4.1 Cómo registrar una decisión nueva	22

Para actualizar: clic derecho sobre el índice → Actualizar campos → Actualizar toda la tabla.

1. Introducción

Este documento reúne las vistas arquitectónicas del sistema y el registro completo de las decisiones de arquitectura adoptadas durante el proyecto.

1.1 Qué es un ADR

Un **Architecture Decision Record** (ADR, registro de decisión de arquitectura) es un documento breve que deja constancia de una decisión técnica relevante: el contexto en que se tomó, la alternativa escogida, su justificación y las consecuencias que acarrea.

El formato responde a un problema concreto: en un proyecto de varios meses, las decisiones se toman en conversaciones y se olvidan. Meses después nadie recuerda por qué el sistema hace las cosas de determinada manera, y se revierten decisiones correctas por desconocimiento del razonamiento original.

Cada ADR de este documento sigue la misma estructura:

Sección	Contenido
Contexto	La situación que obliga a decidir. Incluye las restricciones vigentes
Decisión	Qué se resolvió, enunciado de forma inequívoca
Justificación	Por qué esa alternativa y no otra
Consecuencias	Efectos positivos y negativos de la decisión

La sección de consecuencias negativas no es un formalismo. Una decisión de arquitectura sin costos declarados suele significar que esos costos no se analizaron, no que no existan.

1.2 Índice de decisiones

ID	Decisión	Estado
ADR-001	Arquitectura de microservicios	Aceptada
ADR-002	Un esquema por servicio, no una base por servicio	Aceptada
ADR-003	Comunicación síncrona y asíncrona	Aceptada
ADR-004	Autenticación y propagación del contexto multi-tenant	Aceptada
ADR-005	Origen de los datos de cobros	Aceptada
ADR-006	Dos clientes: aplicación móvil y aplicación web	Aceptada

Ninguna decisión figura como sustituida o revertida. La numeración es correlativa y los identificadores no se reutilizan: si una decisión fuera reemplazada más adelante, se registraría un ADR nuevo indicando cuál sustituye.

2. Vistas de la arquitectura

2.1 Vista de componentes

Muestra los elementos que componen el sistema y la forma en que se comunican entre sí.

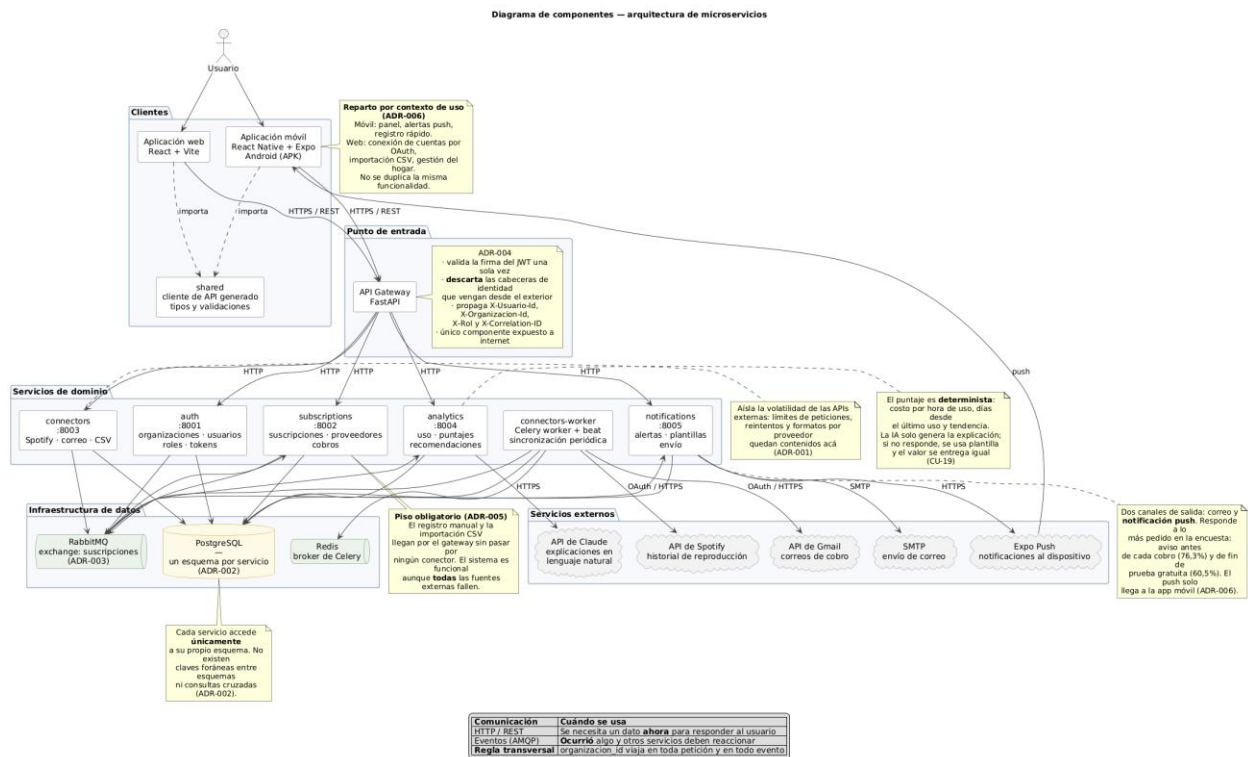


Figura 1. Diagrama de componentes del sistema.

El sistema se organiza en cinco servicios de dominio —autenticación, suscripciones, conectores, análisis y notificaciones— más un API Gateway que constituye el único punto de entrada. Cada servicio dispone de su propio esquema de base de datos y publica eventos hacia los demás mediante un intermediario de mensajería.

Los clientes, aplicación móvil y aplicación web, consumen la misma API a través del gateway.

2.2 Vista de despliegue

Muestra dónde se ejecuta cada componente y cómo se distribuye la infraestructura.

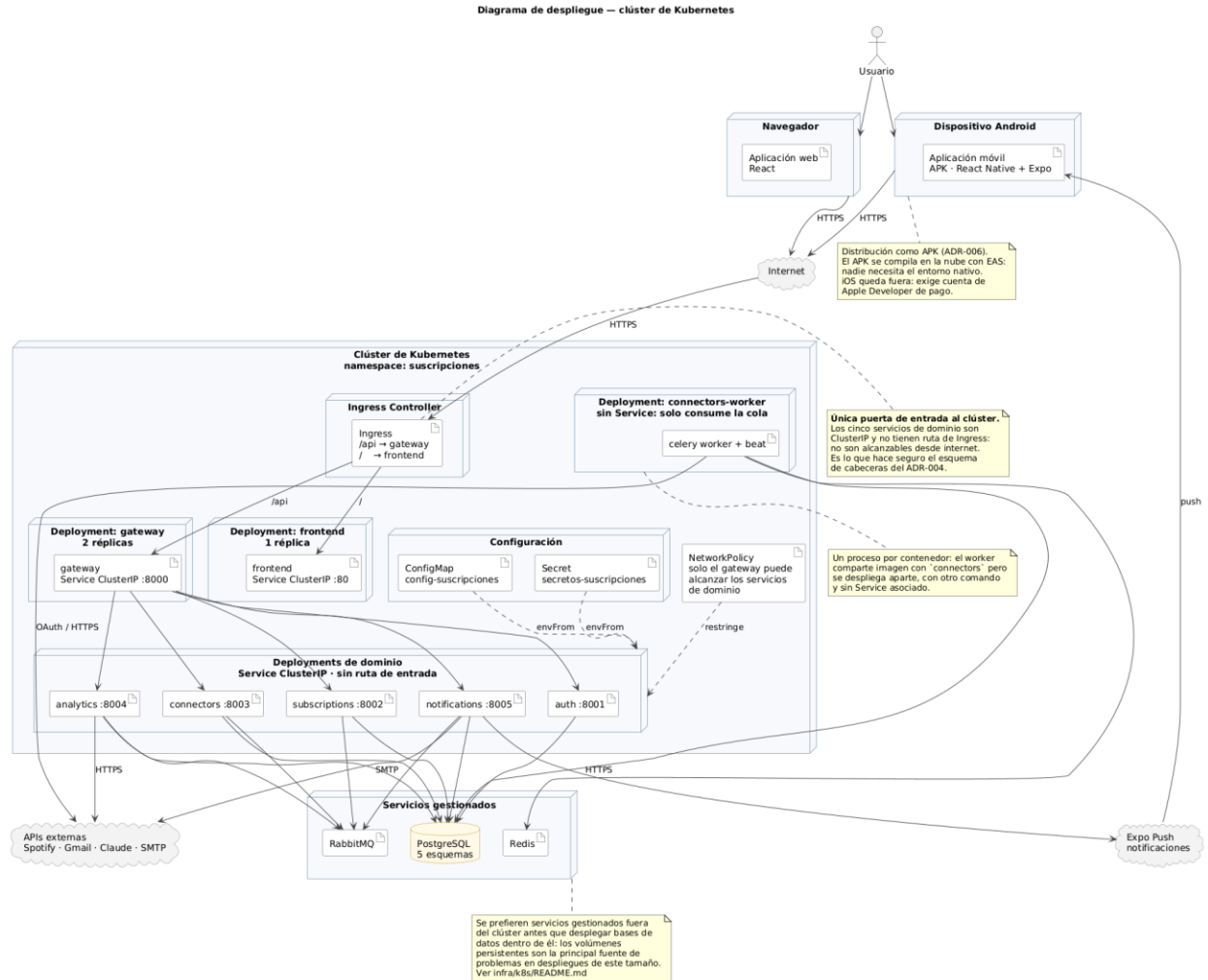


Figura 2. Diagrama de despliegue en el clúster de Kubernetes.

Los servicios de dominio se despliegan sin ruta de entrada desde el exterior: únicamente el gateway y la aplicación web resultan alcanzables desde internet. Esta disposición es la que sustenta el modelo de autenticación descrito en el ADR-004.

2.3 Vista funcional

El modelo de casos de uso comprende veinticinco casos agrupados en cinco paquetes funcionales: gestión de cuenta, gestión de suscripciones, conectores externos, análisis y recomendación, y alertas.

Dada su extensión, se documenta en un artefacto propio —«Modelo de casos de uso»— que incluye el diagrama en notación UML y las narrativas extendidas de los casos de mayor prioridad, con sus flujos alternativos y excepciones.

3. Registro de decisiones de arquitectura

Se transcriben a continuación las seis decisiones registradas, en orden cronológico.

3.1 ADR-001 · Arquitectura de microservicios

Fecha: 2026-08-21 · **Estado:** aceptada

Contexto

El sistema es un SaaS multi-tenant con seis desarrolladores y un semestre de plazo. Se evaluaron dos estilos: monolito modular y microservicios.

Se identificaron componentes con características de operación marcadamente distintas:

- **Conectores externos:** dependen de APIs de terceros inestables, con límites de peticiones y fallos frecuentes. Su tasa de cambio es alta.
- **Análisis y recomendación:** cómputo pesado y periódico, no ligado al ciclo de petición y respuesta.
- **Autenticación:** transversal, de bajo cambio y alta criticidad.
- **Notificaciones:** responsabilidad única, sin dependencias del dominio.

Decisión

Arquitectura de **microservicios** con cinco servicios de dominio más un API Gateway como punto único de entrada.

Justificación

1. **Aislamiento de fallos.** Si la API de Spotify deja de responder, se degrada únicamente `connectors`. El resto del sistema sigue operando: el usuario puede consultar su panel y registrar suscripciones sin verse afectado.
2. **Escalamiento independiente.** `analytics` y `connectors` tienen perfiles de carga distintos al resto y pueden escalarse por separado.
3. **Contención de la volatilidad.** Los cambios de las APIs externas quedan confinados a un servicio y no se propagan al modelo de dominio.
4. **Alineación con la estructura del equipo.** Seis personas, seis componentes. Cada integrante es dueño de uno, con su propio despliegue y sus pruebas. Esto reduce los conflictos de integración, que en equipos de este tamaño son una fuente real de pérdida de tiempo.
5. **Fronteras explícitas.** Cada servicio publica un contrato OpenAPI. Las dependencias entre componentes quedan documentadas y son verificables.

Consecuencias

Positivas

- Fallos aislados por servicio
- Despliegue y escalamiento independientes
- Propiedad clara del código por persona
- Contratos explícitos entre componentes

Negativas — asumidas conscientemente

- **Mayor complejidad operativa.** Seis procesos, un broker de mensajes y una base de datos. Se mitiga con Docker Compose: un solo comando levanta todo.
- **Depuración distribuida.** Un error puede atravesar tres servicios. Se mitiga con un identificador de correlación propagado en cada petición y evento.
- **Consistencia eventual.** Las operaciones que cruzan servicios no son transaccionales. Se acepta porque el dominio lo tolera: que una recomendación se calcule algunos segundos después de registrarse el uso no tiene impacto.
- **Sobrecarga inicial.** Levantar seis esqueletos toma más tiempo que uno solo. Se concentra en la primera semana y se amortiza durante el resto del proyecto.

Alternativa descartada

Monolito modular. Menor complejidad operativa y desarrollo inicial más rápido. Se descartó porque no ofrece aislamiento de fallos frente a las APIs externas — que son el punto más frágil del sistema — ni permite la propiedad independiente del código por integrante.

Riesgo declarado

El riesgo principal es que la complejidad operativa consuma tiempo destinado a funcionalidad.

Mitigación: los servicios notifications y analytics están diseñados para poder fusionarse en uno solo si a mitad del proyecto el avance no es el esperado. Las fronteras están definidas de modo que esa fusión no obligue a rediseñar el resto del sistema.

3.2 ADR-002 · Un esquema por servicio, no una base por servicio

Fecha: 2026-08-21 · **Estado:** aceptada

Contexto

La ortodoxia de microservicios indica **database per service**: cada servicio con su propia instancia de base de datos, sin acceso al almacenamiento de los demás.

En el contexto de este proyecto, cinco instancias de PostgreSQL implican cinco configuraciones, cinco respaldos y cinco despliegues, sobre planes gratuitos de hospedaje que limitan la cantidad de bases de datos disponibles.

Decisión

Un esquema de PostgreSQL por servicio, dentro de una única instancia. La separación lógica es estricta:

- Ningún servicio consulta el esquema de otro. Nunca.
- No existen claves foráneas entre esquemas.
- Los datos que un servicio necesita de otro se obtienen por HTTP o llegan por evento, y se almacenan desnormalizados cuando corresponde.

Justificación

La regla que realmente importa de **database per service** es que ningún servicio lea directamente el almacenamiento de otro. Esa regla se cumple íntegramente con esquemas separados.

Lo que se pierde es el aislamiento físico, que resuelve problemas de escala que este sistema no tiene.

La migración a instancias separadas es posterior y no requiere cambios en el código: basta apuntar cada servicio a una cadena de conexión distinta. Ese es precisamente el argumento — **la decisión es reversible sin costo de rediseño**.

Consecuencias

Positivas

- Una sola instancia que configurar, respaldar y desplegar
- Cabe en los planes gratuitos de hospedaje
- Migración a instancias separadas sin cambios de código

Negativas

- Sin aislamiento físico: una consulta pesada afecta a todos los servicios
- La separación depende de la disciplina del equipo, no del motor

Mitigación: en producción cada servicio se conecta con un usuario propio de PostgreSQL, con permisos únicamente sobre su esquema. Así la separación pasa a estar impuesta por el motor y no por la buena voluntad del desarrollador. Ver `infra/migraciones/01-esquemas.sql`.

3.3 ADR-003 · Comunicación síncrona y asíncrona

Fecha: 2026-08-21 · **Estado:** aceptada

Contexto

Los servicios necesitan intercambiar información. Usar siempre HTTP produce acoplamiento temporal: si un servicio está caído, el que lo llama falla. Usar siempre eventos hace imposible responder consultas inmediatas.

Decisión

Una regla única, fácil de aplicar sin tener que deliberar cada vez:

Si necesito un dato ahora para responderle al usuario → HTTP Si algo ocurrió y otros deben reaccionar → evento

Tipo	Mecanismo	Ejemplo
Síncrona	HTTP/REST a través del gateway	El frontend solicita el panel de gastos
Asíncrona	Eventos publicados en RabbitMQ	Se registró uso, entonces analytics recalcula el puntaje

Justificación

Todo lo que ocurre después de una acción del usuario — recalcular recomendaciones, enviar alertas, sincronizar conectores — es asíncrono por naturaleza. No existe razón para que el usuario espere a que termine.

Las consultas que alimentan una pantalla sí deben ser síncronas: el usuario está esperando la respuesta en ese momento.

Consecuencias

- Los eventos desacoplan: analytics puede estar caído y los eventos quedan encolados hasta que vuelva a estar disponible
- Consistencia eventual entre servicios, aceptable en este dominio
- Los consumidores deben ser **idempotentes**: un evento puede llegar dos veces y el resultado debe ser el mismo
- Cada petición y cada evento lleva un X-Correlation-ID, que permite seguir una operación completa a través de los registros de varios servicios

El catálogo de eventos está en `shared/events/catalogo.md`.

3.4 ADR-004 · Autenticación y propagación del contexto multi-tenant

Fecha: 2026-08-21 · Estado: aceptada

Contexto

Con seis componentes, validar el token en cada uno significa que todos dependen del servicio de autenticación en cada petición, y que la lógica de validación queda replicada seis veces.

Además, el sistema es multi-tenant: **cada servicio debe saber a qué organización pertenece la petición que está atendiendo.**

Decisión

1. auth **emite** los tokens JWT firmados, con `usuario_id`, `organizacion_id` y `rol` en su contenido.
2. El **gateway valida la firma una sola vez** por petición.
3. El gateway propaga la identidad a los servicios internos mediante cabeceras:

```
`` X-Usuario-Id:          7f3a1b...  X-Organizacion-Id:  9c2140...  X-Rol:
admin X-Correlation-ID:  4b8e77... ``
```

1. Los servicios internos **confían en esas cabeceras** y no vuelven a validar.
2. Cada servicio **filtra obligatoriamente** por `X-Organizacion-Id`.

Justificación

- Un solo lugar valida firmas, y por lo tanto un solo lugar donde equivocarse
- Los servicios internos no dependen de auth en cada petición
- El contexto de organización viaja de forma uniforme y explícita por todo el sistema, incluidos los eventos

Consecuencias

⚠ **Riesgo crítico.** Si un servicio interno queda expuesto a internet, cualquiera puede enviar la cabecera `X-Organizacion-Id` que quiera y leer los datos de cualquier cliente.

Mitigaciones obligatorias:

1. Los servicios internos **nunca** se publican fuera de la red interna. Solo el gateway expone puertos al exterior.
2. En producción, los servicios aceptan tráfico únicamente desde el gateway.
3. Las cabeceras de identidad que lleguen desde el exterior son **descartadas por el gateway** antes de reenviar la petición. Un cliente no puede inyectar su propia `X-Organizacion-Id`.

El punto 3 es el que hace que el esquema sea seguro, y **hay que poder explicarlo en la defensa**: sin él, el diseño completo se cae.

La clase base que aplica el filtro en cada servicio está en `shared/tenant/`.

Nota sobre los puertos en desarrollo

El archivo `docker-compose.yml` publica los puertos 8001 a 8005 para facilitar el trabajo y las pruebas.

Esa configuración es exclusiva de desarrollo local. En el despliegue, esos puertos no se publican.

3.5 ADR-005 · Origen de los datos de cobros

Fecha: 2026-08-26 · **Estado:** aceptada **Investigación:** ámbito financiero — acceso a datos de cobros en Chile

Contexto

La funcionalidad más demandada en la encuesta fue «**que me avise antes de cada cobro**» (76,3%, n = 38), seguida por «**que me avise antes de que termine una prueba gratuita**» (60,5%). Ambas exigen que el sistema conozca **cuándo se va a cobrar**.

Eso convierte el origen de los datos de cobro en una decisión de arquitectura, no en un detalle de implementación: **sin fuente de cobros no hay producto**.

Se evaluaron cuatro vías.

1. Las cuatro vías comparadas

Vía	Costo	Esfuerzo	Riesgo legal	Cobertura	Disponible en el semestre
Entrada manual + CSV	\$0	Bajo	Ninguno	Total, pero depende de la constancia del usuario	☒ Sí
Lectura de correo	\$0	Medio-alto	Medio — alcance restringido de Google	Parcial: solo servicios que envían comprobante	☒ Sí, con límites
Agregación bancaria (Fintoc, Belvo)	Alto	Medio	Alto — datos financieros de terceros	Alta	☐ Solo en entorno de prueba
Sistema de Finanzas Abiertas (Ley Fintech)	Por definir	—	Regulado	Alta	☒ No

1.1 Entrada manual e importación CSV

Sin costo, sin dependencias externas y sin exposición regulatoria. Su debilidad es conocida: exige que el usuario mantenga la información al día.

Es el piso obligatorio del sistema. Toda otra vía puede fallar —una API externa se cae, un conector se desautoriza— y la aplicación debe seguir siendo utilizable. Ninguna funcionalidad debe depender exclusivamente de un conector.

1.2 Lectura de correo

Se aborda en detalle en la tarea «Script de lectura de correos». Dos restricciones relevantes para esta decisión:

- **Cobertura parcial.** Solo detecta cobros de servicios que envían comprobante por correo, y el formato varía por proveedor.

- **Alcance restringido de Google.** La lectura del buzón es un permiso sensible. Las aplicaciones en estado de prueba admiten un número acotado de usuarios sin proceso de verificación; publicarlas abiertamente exige una evaluación de seguridad. ⚠ **Las condiciones y límites vigentes deben confirmarse en la consola de Google Cloud antes de comprometer alcance.**

Para un proyecto universitario con usuarios de prueba, el modo restringido es suficiente.

1.3 Agregación bancaria

Ver capítulo 2.

1.4 Sistema de Finanzas Abiertas

Ver capítulo 3.

2. Estado de la agregación bancaria en Chile

Dos proveedores operan en el mercado chileno con producto maduro.

Fintoc

Aspecto	Dato
Cobro	1% + IVA por transacción vía open banking (medio de pago)
Mínimo de facturación	6,5 UF + IVA mensuales, aplicable a todos los productos salvo integraciones de comercio electrónico
Conciliación bancaria	Cobro por cuenta bancaria conectada
Entorno de prueba	Sandbox disponible sin costo
Contratación	Vía equipo comercial

Con la UF a **\$40.867** (26-08-2026), ese mínimo equivale a **≈ \$265.600 mensuales, ≈ \$316.100 con IVA**, se use o no el servicio.

Belvo

Aspecto	Dato
Planes	Desde USD 1.000 mensuales, con planes a medida por volumen
Entorno de prueba	Sandbox gratuito con datos ficticios
Cobertura	Regional; su foco declarado está en México y Brasil

Conclusión del capítulo

La agregación bancaria en producción es económicamente inviable para este proyecto. El piso de gasto mensual de cualquiera de los dos proveedores supera el presupuesto completo de un trabajo

universitario. Los entornos de prueba, en cambio, sí son accesibles y gratuitos. Permiten implementar y demostrar la integración con datos ficticios sin contratar el servicio.

3. Ley Fintech 21.521 y el Sistema de Finanzas Abiertas

La Ley 21.521 regula servicios financieros tecnológicos y crea el **Sistema de Finanzas Abiertas (SFA)**: un ecosistema de intercambio de datos financieros con consentimiento explícito del titular, que permitiría a terceros autorizados acceder a información bancaria mediante interfaces estandarizadas.

Es, en teoría, la solución ideal al problema de este ADR.

El cronograma

Hito	Fecha
Publicación de la ley	2023
Entrada en operación originalmente prevista	Julio de 2026
Consulta pública de la CMF que propone posponer 12 meses	Noviembre de 2025
Entrada en operación según NCG 569	Julio de 2027
Sandbox y Directorio de la CMF disponibles	9 meses antes → ≈ octubre de 2026

Tras la entrada en vigencia rige además un calendario gradual: entre 5 y 18 meses para bancos y emisores de tarjetas según el tipo de interfaz, y entre 20 y 30 meses para el resto de las entidades reguladas.

Conclusión del capítulo

⚠ El Sistema de Finanzas Abiertas no estará operativo durante este proyecto, ni durante el año siguiente. La postergación de la CMF lo lleva a julio de 2027, y las interfaces de mayor utilidad para este caso llegan aún después. Esto no es un obstáculo, es un argumento de diseño: la arquitectura de conectores (services/connectors) permite incorporar el SFA como una fuente adicional cuando exista, sin rediseñar el sistema. El proyecto queda deliberadamente preparado para una capacidad que el mercado todavía no ofrece.

4. Decisión

Se adopta una estrategia escalonada:

Prioridad	Fuente	Estado en el proyecto
1	Entrada manual + importación CSV	Implementada. Piso obligatorio: el sistema funciona sin ningún conector
2	Lectura de correo	Implementada con usuarios de prueba
3	Agregación bancaria	Integración contra el sandbox de Fintoc, si el tiempo lo permite. Producción fuera de alcance por costo

Prioridad	Fuente	Estado en el proyecto
4	Sistema de Finanzas Abiertas	Trabajo futuro, con fecha conocida: julio de 2027

Justificación

- 1. El producto funciona desde el primer día.** Las dos funcionalidades más demandadas —aviso de cobro y aviso de fin de prueba— se satisfacen con entrada manual. No dependen de ninguna integración externa.
- 2. El costo de la agregación bancaria excede el proyecto por dos órdenes de magnitud.** No es una limitación de diseño, es una restricción de mercado documentada.
- 3. La integración es demostrable sin contratarla.** El sandbox permite mostrar el conector funcionando en la defensa.
- 4. La arquitectura no queda comprometida.** El contrato común de conectores (`sincronizar()`, `obtener_uso()`) admite una fuente nueva sin tocar el resto del sistema.

Consecuencias

Positivas

- Ningún riesgo de que el proyecto quede bloqueado por una dependencia externa
- Sin exposición regulatoria en producción
- Ruta de evolución documentada y fechada

Negativas — asumidas

- La detección automática de cobros queda limitada a lo que el correo permita
- La carga inicial de suscripciones recae en el usuario, lo que constituye fricción de adopción real y debe declararse en el informe

5. Protección de datos — Ley 21.719

Vigencia

La Ley 21.719 fue publicada el **13 de diciembre de 2024** y **entra en vigencia el 1 de diciembre de 2026**, es decir, **durante el desarrollo de este proyecto**.

Obligaciones relevantes

Obligación	Qué implica acá
Base de licitud y consentimiento	El consentimiento debe ser específico, informado y revocable. Conectar una cuenta externa exige consentimiento propio y separado
Derechos ARCO	Acceso, rectificación, cancelación y oposición. El usuario debe poder exportar y eliminar sus datos

Obligación	Qué implica acá
Registro de actividades de tratamiento	Documentar qué datos se tratan, para qué y por cuánto tiempo
Notificación de brechas	Reporte a la Agencia dentro de 72 horas
Datos financieros	Si la brecha involucra datos financieros, además hay que comunicar a los titulares afectados en lenguaje claro
Sanciones	Hasta 20.000 UTM, o 4% de los ingresos anuales en caso de reincidencia

Consecuencias de diseño

Estas obligaciones se traducen en decisiones concretas, no en una sección declarativa del informe:

1. **Minimización.** Se almacena únicamente proveedor, monto, fecha y ciclo. **No se guarda la cartola completa ni el detalle de transacciones ajenas a suscripciones.** Menos datos almacenados, menor exposición ante una brecha.
2. **Cifrado de credenciales.** Los tokens de los conectores se almacenan cifrados y nunca se exponen en la interfaz ni en los registros de la aplicación (ya establecido en CU-12).
3. **Consentimiento separado y revocable por conector.** Aceptar los términos de la aplicación no habilita a leer el correo. Desconectar un conector debe **eliminar** los datos obtenidos por esa vía, no solo revocar el acceso.
4. **Retención acotada.** Definir y documentar por cuánto tiempo se conservan los registros de uso.
5. **Aislamiento verificable.** El multi-tenancy de shared/tenant deja de ser solo una decisión técnica: es la medida que impide que los datos de un hogar sean accesibles a otro.

Una obligación que ya aplica hoy

La planilla de la encuesta contiene **16 correos electrónicos** de personas que se ofrecieron a probar la aplicación. Eso constituye tratamiento de datos personales, con independencia de que el software aún no exista.

Medidas adoptadas:

- La carpeta docs/ está excluida del repositorio público (ver .gitignore)
- Esos correos se usan **únicamente** para el fin declarado al recogerlos
- Deben eliminarse al término del proyecto si no se concreta la invitación

Fuentes consultadas

Todas verificadas el **26 de agosto de 2026**. Las condiciones comerciales y los plazos regulatorios cambian: **reverificar antes de la entrega final.**

- CMF — [Norma que regula el Sistema de Finanzas Abiertas](#)

- CMF — [Consulta pública sobre modificaciones al SFA](#)
- CMF — [Claves de la actualización de la NCG 514 \(PDF\)](#)
- Cuatrecasas — [NCG 569 y reglas técnicas del SFA](#)
- Carey — [Propuesta que posterga 12 meses la entrada en vigencia del SFA](#)
- Fintoc — [Tarifas y comisiones](#)
- Belvo — [Planes y precios](#)
- Belvo — [Documentación del sandbox](#)
- Thomson Reuters — [Ley 21.719 y la reconstrucción del derecho chileno de protección de datos](#)
- Conformer360 — [Ley 21.719 en banca y servicios financieros](#)
- Valor de la UF al 26-08-2026 — [calcular.cl](#)

3.6 ADR-006 · Dos clientes: aplicación móvil y aplicación web

Fecha: 2026-08-31 · **Estado:** aceptada **Sustituye la decisión previa de un único cliente web responsive**

Contexto

El ramo exige la entrega de una **aplicación móvil**. La decisión anterior —una sola aplicación web responsive que cubriera computador y celular— no satisface ese requisito.

Se define además construir **ambos clientes**: aplicación móvil y aplicación web.

Datos de la encuesta que condicionan la decisión:

- El **84,2%** de los encuestados utilizaría la solución en el teléfono móvil
- El **47,4%** en computador
- La funcionalidad más demandada es el **aviso previo a cada cobro (76,3%)**, seguida por el aviso de término de prueba gratuita (60,5%)

Decisión

Dos clientes que consumen la misma API, con funcionalidades repartidas según el contexto de uso. No se construye la misma aplicación dos veces.

Cliente	Tecnología	Distribución
Móvil	React Native con Expo (TypeScript)	APK de Android
Web	React con Vite (TypeScript)	Desplegada en el clúster

Reparto de funcionalidades

Funcionalidad	Móvil	Web
Panel de gastos	Vista resumida	Vista completa
Notificaciones push	Exclusivo	—
Registrar suscripción	Sí	Sí
Marcar uso manual	Sí	—
Conectar cuentas externas (OAuth)	—	Exclusivo
Importar CSV	—	Sí
Gestionar integrantes del hogar	—	Sí
Recomendaciones e informes	Resumen	Detalle

Código compartido

Ambos clientes están escritos en TypeScript, por lo que comparten un paquete real y no solamente un contrato:

```
`` frontend/ └─ shared/           cliente de API generado desde OpenAPI, |
tipos, validaciones y lógica de presentación |           (formato de montos,
cálculo de días al próximo cobro) └─ web/           React + Vite └─ movil/
React Native + Expo ``
```

El backend expone su esquema mediante drf-spectacular; de ahí se genera el cliente TypeScript que ambas aplicaciones importan. Un cambio de contrato en el backend se detecta al compilar, en los dos clientes, y no en ejecución.

Justificación

Por qué el reparto por contexto y no funcionalidad duplicada

Duplicar toda la funcionalidad implica construir dos veces la capa que más horas consume del proyecto. El reparto por contexto reduce el trabajo total y responde a cómo se usa realmente cada dispositivo.

Notificaciones push exclusivas del móvil. Es la capacidad que una aplicación nativa aporta y una web no entrega con fiabilidad. Coincide con la funcionalidad más demandada del estudio.

OAuth exclusivo de la web. Vincular una cuenta externa desde móvil exige **deep linking**, mecanismo frágil y costoso en tiempo de desarrollo. En web es un redireccionamiento convencional. El usuario vincula sus cuentas una vez, en una sesión de configuración, y luego opera desde el teléfono en el día a día. La decisión simplifica la implementación y coincide con el comportamiento real.

Por qué React Native y no Flutter

Se evaluó Flutter como alternativa. Se descartó por tres razones acumulativas:

1. **Un solo lenguaje en todo el frontend.** El equipo ya trabaja con React y TypeScript en el cliente web. React Native conserva el lenguaje, los conceptos y el razonamiento. Flutter implicaría incorporar Dart, nuevo para todos los integrantes, dentro del mismo semestre.
2. **Código efectivamente compartido.** Con ambos clientes en TypeScript, el paquete shared contiene código ejecutable —cliente de API, tipos, validaciones, formateo— y no únicamente un contrato del que cada lado genera su propia versión. Con Flutter esa reutilización no existe.
3. **Movilidad dentro del equipo.** Quien desarrolla la web puede apoyar el móvil y viceversa. Con dos lenguajes distintos, cada persona queda encerrada en su cliente, lo que es un riesgo real en un equipo de seis con plazos fijos.

Por qué Expo

React Native sin Expo requiere que cada integrante instale y configure Android Studio, el SDK de Android, Java y Gradle. Es una fuente conocida de pérdida de tiempo y falla de manera distinta en cada máquina.

Expo elimina esa barrera:

- La aplicación se ejecuta **en el teléfono real mediante un código QR**, sin emulador ni entorno nativo instalado

- La compilación del APK ocurre en la nube, sin toolchain local
- Incorpora los módulos que el proyecto necesita: notificaciones push (expo-notifications), almacenamiento seguro de credenciales (expo-secure-store) y sesión de autorización (expo-auth-session)

Por qué no una sola aplicación para ambas plataformas

Se descartó usar React Native Web para cubrir también el navegador. El resultado se comporta como una aplicación móvil ampliada, con un modelo de accesibilidad y un rendimiento de carga inferiores a los de una web convencional. Para un panel de escritorio con tablas e importación de archivos, React con Vite es más adecuado.

Consecuencias

Positivas

- Se satisface el requisito del ramo
- Las notificaciones push refuerzan la propuesta de valor central del producto
- Un solo lenguaje en todo el frontend, con código realmente compartido
- El contrato con la API queda verificado por el compilador en ambos clientes
- La complejidad del OAuth queda contenida en un solo cliente

Negativas — asumidas conscientemente

- Dos bases de código de interfaz que mantener
- Dos procesos de despliegue distintos: la web al clúster, el móvil como archivo distribuible
- React Native introduce diferencias respecto de React web —navegación, estilos, componentes— que requieren aprendizaje aunque el lenguaje sea el mismo

Limitación declarada: solo Android

La entrega se realiza como **archivo APK para Android**. Se descarta iOS porque la distribución requiere una cuenta de Apple Developer de costo anual, no contemplada en el presupuesto del proyecto.

Debe declararse explícitamente en el informe como limitación de alcance.

Notificaciones push

El envío se realiza mediante el **servicio de notificaciones de Expo**. El servicio notifications pasa a gestionar **dos canales de salida**: correo electrónico y push.

⚠ **El push nunca es el único canal.** Si el usuario no tiene la aplicación instalada, o la entrega falla, la alerta se envía igualmente por correo. La funcionalidad no depende de que el dispositivo esté disponible.

Plan de contingencia

Si a mitad del semestre el avance no permite completar ambos clientes, se recorta la aplicación web, no la móvil.

Fundamento: la aplicación móvil es requisito del ramo, y el 84,2% de los encuestados declara que utilizaría la solución en el teléfono.

Esta decisión se toma **por anticipado y por escrito**, de modo que no deba discutirse bajo presión de plazo.

Impacto sobre decisiones anteriores

Documento	Cambio
ADR-001	Sin cambios. La arquitectura de microservicios no se ve afectada
ADR-004	Sin cambios. Ambos clientes se autentican contra el mismo gateway
CU-12, CU-13	El flujo OAuth se restringe al cliente web
CU-23, CU-24	Las notificaciones incorporan el canal push además del correo
Servicio notifications	Gestiona dos canales de salida

4. Trazabilidad y mantención

Los archivos fuente de este documento residen en el repositorio del proyecto y constituyen la versión autoritativa. Este documento es una consolidación para efectos de entrega.

Artefacto	Ubicación
Decisiones de arquitectura	docs/arquitectura/ADR-*.md
Diagramas (fuente editable)	docs/diagramas/*.puml
Diagramas (imágenes)	docs/diagramas/png/
Script de renderizado	docs/diagramas/render.py
Glosario del dominio	docs/glosario.md
Modelo de casos de uso	docs/diagramas/casos-de-uso-narrativas.md

⚠ Los diagramas se generan a partir de sus archivos fuente mediante `render.py`. Al modificar un diagrama debe regenerarse la imagen y actualizarse este documento; de lo contrario, texto e imagen dejan de coincidir.

4.1 Cómo registrar una decisión nueva

1. Crear `docs/arquitectura/ADR-00N-titulo-breve.md`, con N correlativo. Los identificadores no se reutilizan.
2. Seguir la estructura de contexto, decisión, justificación y consecuencias.
3. Declarar las consecuencias negativas. Si no se identifica ninguna, la decisión no fue analizada en profundidad.
4. Si sustituye a una decisión anterior, indicarlo en la cabecera de ambos documentos.
5. Actualizar los diagramas afectados y regenerar las imágenes.